



## TECHNICAL OVERVIEW

---

# Platform Architecture

## The Corello Knowledge Layer

The knowledge layer that sits between a manufacturer's own data and a language model, and the agents built on top of it.

**5,000+**

KNOWLEDGE NODES  
SINGLE DEPLOYMENT

**4,000+**

MAPPED  
RELATIONSHIPS

**Zero**

ONTOLOGY CONFIGURATION  
REQUIRED

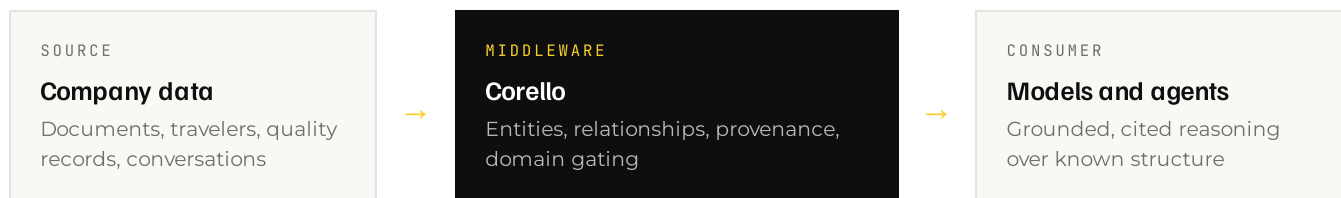
CORE CONCEPT

**Corello is a middleware knowledge layer. It sits between a company's own data and a general purpose language model, and it tells the model what that company's data actually means.**

The knowledge that runs a manufacturing operation is largely unwritten. It exists as relationships between concepts: which process suits which material, which customer tolerates which substitution, why a given step exists at all. That knowledge is held by experienced people and recorded nowhere. Corello captures the structure and makes it queryable.

This is what a general purpose model cannot do unaided. A frontier model has never been trained on a given manufacturer's data, and that data is not something the manufacturer will hand over for training. Asked to reason about it directly, the model produces plausible output that field staff cannot distinguish from fabrication, and verification costs more than the answer saves. Corello resolves this by supplying the semantic layer, so the model reasons over an explicit, cited knowledge structure rather than guessing at unfamiliar material.

Position in the stack



Platform characteristics

|                                                        |                                          |                                                   |                                                                 |
|--------------------------------------------------------|------------------------------------------|---------------------------------------------------|-----------------------------------------------------------------|
| <b>5,000+</b><br>KNOWLEDGE NODES,<br>SINGLE DEPLOYMENT | <b>4,000+</b><br>MAPPED<br>RELATIONSHIPS | <b>Zero</b><br>ONTOLOGY CONFIGURATION<br>REQUIRED | <b>&lt;1 day</b><br>TO STAND UP A NEW<br>USE CASE ONCE CAPTURED |
|--------------------------------------------------------|------------------------------------------|---------------------------------------------------|-----------------------------------------------------------------|

## 02

## PART ONE

## Platform Architecture

The core platform is written in .NET Core and runs on Azure Kubernetes Service across three environments. Every component is containerised and the whole infrastructure is declared as code, making the platform portable to on premise, GCP, or AWS without redesign. Build and deployment are automated end to end, so a single commit reaches production in minutes.

### Architectural decisions

- 01 Layered architecture rather than premature microservices.** The platform uses an onion architecture in which each logical service is already isolated and communicates through the same contract. Microservices add operational overhead until scale demands them. Because every service is written the same way, decomposition is available whenever it becomes worthwhile rather than paid for upfront.
- 02 No lock in at any data layer.** The vector store, graph store, and language models are all replaceable. Each is treated as an interchangeable component, so the platform can adopt better technology as it appears instead of being anchored to present choices.
- 03 API first and MCP first throughout.** Every piece of business logic is exposable as a service or an MCP endpoint. External tooling connects without adaptation, and customer specific agents are assembled from existing capability rather than built bespoke.
- 04 Granular permissioning at the knowledge layer.** Access is controlled per user and per privilege over the knowledge base itself, not only at the application boundary. This is what makes the platform deployable inside an organisation where different roles must see different subsets of the same knowledge.
- 05 Security designed in rather than retrofitted.** The platform was built security first under technical leadership drawn from enterprise cybersecurity, and an independent assessment confirmed the implementation follows established security criteria. Corello deploys inside customer environments that are themselves SOC 2 and HIPAA compliant, and full certification of the platform is in progress.
- 06 On premise and sovereign cloud deployment.** Because containerisation and infrastructure as code were foundational, the same platform runs entirely within a customer facility. Deployment onto Microsoft's government community cloud is also being pursued, for environments where data cannot transit commercial infrastructure.

## Data layer, with every component interchangeable

| LAYER             | CURRENT IMPLEMENTATION             | STATUS       |
|-------------------|------------------------------------|--------------|
| Application core  | .NET Core, onion architecture      | FOUNDATIONAL |
| Orchestration     | Kubernetes on Azure AKS            | PORTABLE     |
| Vector store      | Qdrant                             | SWAPPABLE    |
| Graph store       | Neo4j                              | SWAPPABLE    |
| Extraction models | Qwen 2.5, small parameter class    | SWAPPABLE    |
| Model serving     | vLLM; ONNX with BentoML            | SWAPPABLE    |
| Infrastructure    | Declarative infrastructure as code | PORTABLE     |

## Quality controls

Because code is generated against written specification, automated testing is produced at volume as a by product of development. Thousands of tests exercise each unit of logic before a human reviews it. A dedicated evaluation harness runs against the live production stack rather than mocked components, using a curated question set across six reasoning categories including multi step retrieval and correct abstention, validated against two independent ground truth providers. Model versions are benchmarked release over release, so quality changes are measured rather than assumed.

## 03

## PART TWO

## The Knowledge Layer

Knowledge capture runs in two phases. Ingestion establishes the initial structure, and interaction extends it indefinitely.

### Phase one, ingestion

---

Source documents are supplied by drag and drop. The system parses them, identifies the entities inside them, and populates the knowledge map progressively as it works. No schema is declared and no mapping is configured. Material typically includes travelers, quality records, certifications, procedures, work instructions, and historical quotations.

- A parsing pipeline handling standard PDF documentation reliably and at quality.
- A multimodal enrichment stage in which every extracted image is described by a vision model.
- A document classifier identifying technical drawings as a distinct class, with extraction routing specific to document type built into the schema.
- Native handling of Word, Excel, and PowerPoint documents in the deployed container.
- Ingestion of paper based shop floor records, including travelers that accompany an order from RFQ through to shipment.

### Phase two, learning through interaction

---

Ingestion is the beginning of capture, not the end of it. As people query the knowledge base and build agents on top of it, the system observes those interactions and derives new entities and new relationships from them. Corrections are the strongest signal. When a user tells an agent its output is wrong and explains why, that explanation becomes part of the map and conditions every subsequent answer.

Enrichment also runs proactively. Where the system identifies a gap or low confidence in what it holds, it asks a targeted question of the right person, and the answer is moderated before entering the graph. Capture requires no change to how anyone works, since a coworker copied on an ordinary email thread is enough.

### Automatic ontology generation

---

The system constructs its own ontology from the source material. This is the platform's principal technical differentiator and the subject of a pending patent.

### Why an ontology defined in advance fails on tribal knowledge

The established pattern, used by large data integration platforms, requires an ontology to be discovered and specified before ingestion can begin, which is a lengthy exercise in itself. It is also structurally unsuited to unwritten knowledge. An ontology defined in advance encodes assumptions about how people describe their work, and those assumptions fail on a shop floor where staff may be working across four languages. Deriving structure from the material sidesteps both problems.

## Graph structure and domain gating

Captured knowledge is held as concepts and the relationships between them, connecting the dots rather than storing documents. A single production deployment holds over 5,000 nodes and over 4,000 relationships. The graph is partitioned into knowledge domains, and each domain can be independently activated or deactivated.

- 01 Domain partitioning is a control surface, not a display feature.** Because domains can be switched off, the operator determines exactly which subset of knowledge is available to any given consumer. Organisational knowledge can be withheld from an external model while process knowledge is shared with it.
- 02 This yields guardrails at the semantic layer.** Filtering happens on meaning rather than on text patterns. Concepts belonging to a disabled domain are not transmitted at all, which is a stronger guarantee than attempting to redact them downstream.
- 03 Hybrid graph and vector storage.** Knowledge is held in graph and vector form together, supporting both semantic search and structured relationship traversal over one body of knowledge. Live operational data access is on the roadmap, extending the same reasoning layer to current plant state.
- 04 Small models for extraction.** Extraction runs on small models, keeping token cost low and on premise deployment viable while holding quality at the level the work requires. Larger models serve retrieval selectively, and any model can be swapped.

## Query interface and provenance

A chat interface allows the knowledge base to be queried directly. Answers carry their sources, so a user can see which document a statement came from and follow the reasoning that produced it. In an environment where the alternative is an unverifiable model output, traceability is what makes the answer usable, and it is possible here because the knowledge structure is explicit rather than latent in a set of weights.

## Under evaluation: live feed capture

Small vision models can now describe a live video feed in real time. Corello is evaluating that channel as a third capture track, covering equipment serial numbers, workstation configuration, and process sequence, requiring neither documentation nor interview.

## 04

## PART THREE

## The Agent Layer

The knowledge layer is not the deliverable. What is built on top of it is. Agents are deployed as coworkers. Each has a working email address, and the people who use it are not asked to adopt anything new.

## The coworker pattern

- 01 The interface is whatever channel already exists.** A request is forwarded to the agent's address and the response comes back the same way. The pattern is channel independent, covering email, Teams, or Slack, because the agent is addressed rather than operated.
- 02 Zero change management by design.** Standard users see no new system. This is a deliberate architectural constraint rather than a preference about interface. In operations where a digital transformation programme is not viable, the technology has to adapt to the existing process instead of the reverse.
- 03 Administration is separated from use.** An admin portal exposes agent activity, outcomes, and configuration for the small number of people who need it. Everyone else interacts with the agent as a colleague and never encounters the platform.
- 04 Every interaction feeds back into the graph.** Responses are answered on one side and captured on the other. A correction supplied in a reply, such as a cost input that has moved or a constraint the agent did not account for, enters the knowledge map and applies from then on.

## Reference implementation: the quoting agent

Quoting is the first agent built on the pattern. An incoming RFQ is forwarded to the agent, which returns a complete response, including the priced quote, in about a minute.

| MANUAL PROCESS                                                               |   | WITH THE AGENT                                                                         |
|------------------------------------------------------------------------------|---|----------------------------------------------------------------------------------------|
| <b>6 to 10 days</b><br>Typical turnaround in high mix, low volume production | → | <b>~2 minutes</b><br>Observed response time, or 6 hours where human review is retained |

- 01 The pricing decision is deterministic, not generative.** Quoting runs on a trained machine learning model served through ONNX and BentoML. Language models interpret the incoming request, but the priced output is reproducible, which is what makes it auditable.
- 02 Output is validated against the knowledge graph.** The model produces a quote, and the knowledge layer then checks it against unstructured context the model cannot see, such as a contract term that invalidates the price or a process constraint specific to that customer. Accuracy improves through the interaction of the two systems rather than through model size.

- 03 Model checkpoints held per customer.** Each deployment carries its own checkpoint reflecting how that operation actually prices work. Versions are benchmarked against one another so change is measured.
- 04 Shadow mode before autonomy.** Agents run alongside the people currently doing the work and are refined against real decisions before assuming any of it.

## Other implementations on the same pattern

- **Onboarding and training.** Captured knowledge is queried directly by new staff, replacing the practice of shadowing an experienced operator to acquire it.
- **Work instruction generation.** Ingested shop floor records inform new work instructions, so current guidance reflects what actually happened on previous runs rather than what was originally specified.

## The meta agent

### Corello as a meta product

Rather than hand build an agent per use case, the knowledge layer generates it. Because the system already holds how a given operation works, it can author the prompts and structure that define a working agent for that operation, in the same relationship a coding agent has to the software it produces. The knowledge layer is therefore a product that makes products, and standing up a new use case once knowledge is captured takes under a day.

## Interface roadmap

The coworker pattern is deliberately independent of any single surface, which leaves the presentation layer open. Work in progress covers shop floor screens and smart glass output for staff who are not at a desk, and multilingual interaction so that a worker is addressed in their own language rather than the language the system was configured in. Agent composition follows the same trajectory. Capacity management informs quoting, quoting adjusts to available capacity, and operational instruction draws on both.

- 1 Quoting**  
Deployed, with deterministic pricing validated against the knowledge graph.
- 2 Capacity management**  
Interacts with quoting to allocate available capacity.
- 3 Operational instructions**  
Shop floor execution guidance drawn from captured process knowledge.
- 4 Composed quote to cash**  
Agents operating together across the operational cycle.



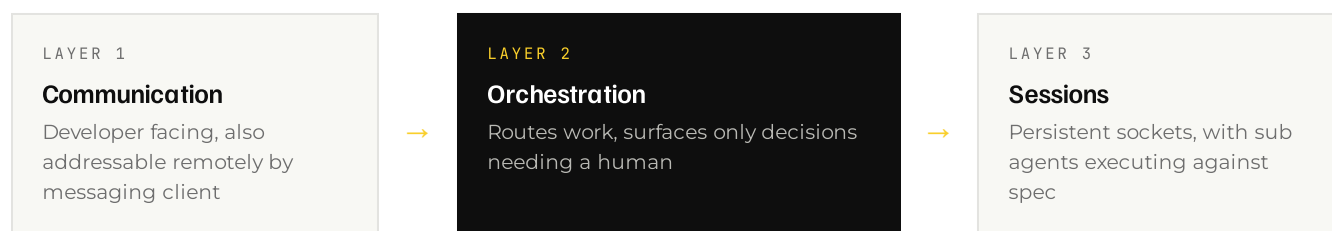
## 05 PART FOUR AstroForge

AstroForge is Corello's internal development platform. It is not customer facing. It exists because the constraint on the product is not architectural but the rate at which the team can specify, build, and validate, and because the same knowledge capture technology the platform sells turns out to apply directly to software engineering.

### Architecture

AstroForge orchestrates many concurrent coding sessions. Each session runs in a persistent socket, which means it survives independently of whatever anyone happens to be looking at. Sessions continue rather than being suspended when attention moves elsewhere, and they are grouped into profiles so that work can be organised by meaning rather than by process.

Above the sessions sits a layer of agents with separated responsibilities. A communication agent handles interaction with the developer. An orchestration agent is responsible only for keeping sessions productive and for deciding what requires human input. The session agents below them do the actual work. Because responsibility is partitioned this way, a developer converses with one agent rather than supervising twenty.



### Astral Code, the specification framework

Every session runs inside a framework that enforces two properties.

- 01 Specification precedes code.** Milestones and phases are defined before implementation begins. Nothing exists outside written specification, which has a secondary benefit. Because intent is recorded rather than held in a model's context, nothing is lost when a session ends and context does not need to be reconstructed.
- 02 Concurrent work is coordinated rather than merged after the fact.** The framework maintains a dedicated repository recording who is doing what and who intends to do what. When two developers specify overlapping work, the system detects it at specification time and raises it before either has written anything. Collisions are prevented rather than resolved.
- 03 Testing is a by product of specification.** Because behaviour is described formally before implementation, tests are generated at volume, with thousands exercising a single unit of logic before a human reviews anything. Human review begins from a validated baseline rather than an unverified one.

- 04 Scale without supervision cost.** The combination supports 20 to 30 projects running in parallel with quality held constant. The limiting factor becomes the developer's decision bandwidth rather than their attention, and the orchestration layer is specifically designed to conserve it.

## Developer persona graphs

AstroForge reads its own sessions and extracts principles and patterns from them using the same mechanism Corello applies to customer documents. Accepted extractions accumulate into a personal knowledge graph for each developer, a structured account of how that engineer works.

- 01 Persona graphs are importable.** A developer can load a colleague's graph and ask their orchestrator to evaluate a specification against it. The system surfaces where two engineers would approach the same problem differently, which converts an architectural disagreement from something discovered in review into something raised at specification time.
- 02 It addresses the collaboration penalty directly.** Engineers working effectively with AI tooling become individually fast enough that coordination becomes the bottleneck, so collaboration slows them down. Combining collision detection at specification time with importable personas removes most of that cost, so senior engineers work the same codebase in parallel without friction.
- 03 It raises the ceiling for non engineers.** Because the framework enforces specification and quality independently of who is writing, team members who understand the domain but not the codebase produce output approaching engineering standard. The population able to respond to a technical requirement widens beyond the engineering team.
- 04 Capability compounds with use.** The graphs mean the system encodes the judgment of the team's strongest engineers and applies it to new specifications. Development capability improves as a function of use, on the same mechanism as the customer facing product.

### The recursion is the point

AstroForge is Corello's own technology turned on Corello. The tribal knowledge of the engineering team, meaning how they specify, what patterns they trust, and where they disagree, is captured, graphed, and put back to work on the next specification. It is the platform's clearest working proof, since the thesis is validated internally before it is sold.

Three layers, each independent of the one below it. Enterprise infrastructure that runs anywhere. A knowledge layer that derives its own structure and gates what leaves it. An agent layer that reaches people through channels they already use. And a development platform built on the same principle as the product it builds.

---

**DOCUMENT PREPARED BY****Andrea Ridi, PhD**

CHIEF TECHNOLOGY OFFICER, CORELLO

**Carlos Maiguel**

CHIEF EXECUTIVE OFFICER, CORELLO

With technical review by an **AI Fellow** provided by C10 Labs.

FOUNDING TEAM · JUNE 2026